

异常处理

复习

1. 计算器概述
2. 计算器抽象基类
3. 计算器子类
4. main函数实现
5. 多文件形式

目录

- | | |
|----------------|----|
| 1. 异常处理概述 | 了解 |
| 2. 标准异常 | 熟悉 |
| 3. try/catch语句 | 重点 |
| 4. 抛出异常 | 重点 |
| 5. 捕获异常 | 重点 |
| 6. 自定义异常 | 重点 |
| 7. 嵌套类和局部类 | 熟悉 |

异常处理概述

```
#include <iostream>
using namespace std;
int main()
{
    int num1 = 100, num2 = 0, num3 = 0;
    num3 = num1 / num2;
    cout << num3 << endl;

    //do something
    return 0;
}
```



这段代码有什么问题吗？

异常处理概述

C++中的异常处理是处理运行时错误的过程。执行异常处理，以便即使在运行时出现错误后也能保持正常的应用程序流转。

在C++中，异常是在运行时抛出的事件或对象。所有异常都派生自 `std::exception` 类。它是一个可以处理的运行时错误。如果我们不处理异常，它就会打印异常消息并终止程序。

异常处理概述

异常提供了一种转移程序控制权的方式。

C++ 异常处理涉及到三个关键字：**try**、**catch**、**throw**。

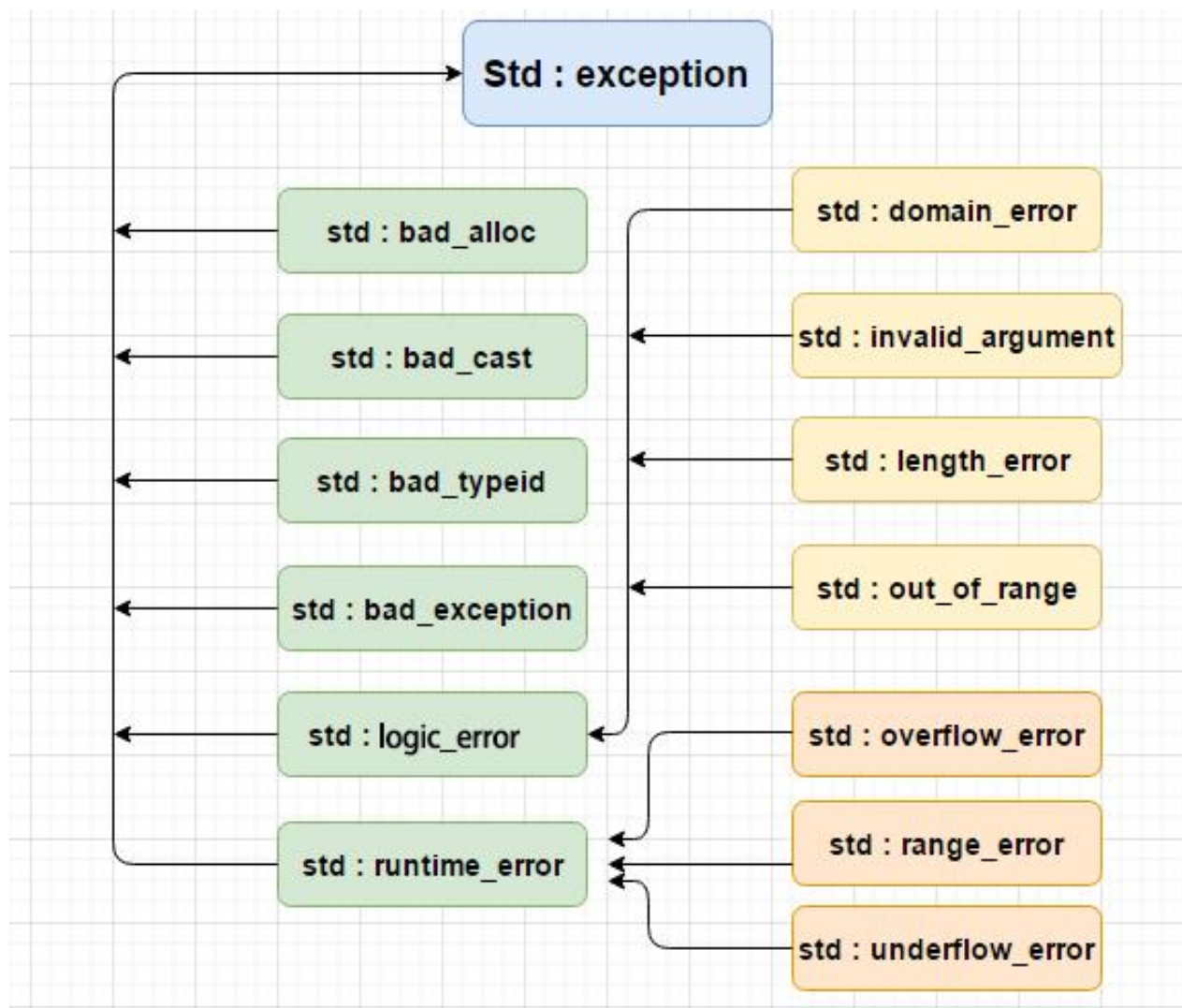
try: try 块中的代码标识将被激活的特定异常。它后面通常跟着一个或多个 catch 块。

catch: 在您想要处理问题的地方，通过异常处理程序捕获异常。catch 关键字用于捕获异常。

throw: 当问题出现时，程序会抛出一个异常。这是通过使用 throw 关键字来完成的。

标准异常

C++ 提供了一系列标准的异常，
定义在 `<exception>` 中，
可以使用这些标准的异常。
它们是以父子类
层次结构组织起来的



标准异常

异常	描述
<code>std::exception</code>	该异常是所有标准 C++ 异常的父类。
<code>std::bad_alloc</code>	该异常可以通过 <code>new</code> 抛出。
<code>std::bad_cast</code>	该异常可以通过 <code>dynamic_cast</code> 抛出。
<code>std::bad_typeid</code>	该异常可以通过 <code>typeid</code> 抛出。
<code>std::bad_exception</code>	这在处理 C++ 程序中无法预期的异常时非常有用。
<code>std::logic_error</code>	理论上可以通过读取代码来检测到的异常。
<code>std::domain_error</code>	当使用了一个无效的数学域时，会抛出该异常。
<code>std::invalid_argument</code>	当使用了无效的参数时，会抛出该异常。
<code>std::length_error</code>	当创建了太长的 <code>std::string</code> 时，会抛出该异常。
<code>std::out_of_range</code>	该异常可以通过方法抛出，例如 <code>std::vector</code> 和 <code>std::bitset<>::operator[]()</code> 。
<code>std::runtime_error</code>	理论上不可以通过读取代码来检测到的异常。
<code>std::overflow_error</code>	当发生数学上溢时，会抛出该异常。
<code>std::range_error</code>	当尝试存储超出范围的值时，会抛出该异常。
<code>std::underflow_error</code>	当发生数学下溢时，会抛出该异常。

try/catch语句

try 块中放置可能抛出异常的代码，try 块中的代码被称为保护代码（有可能出错的代码）。如果有一个块抛出一个异常，捕获异常的方法会使用 try 和 catch 关键字。

使用 try/catch 语句的语法如下所示：

```
try
{
    // 保护代码（有可能出错的代码）
} catch( ExceptionName e1 )
{
    // catch 块
} catch( ExceptionName e2 )
{
    // catch 块
} catch( ExceptionName eN )
{
    // catch 块
}
```

try 块在不同的情境下会抛出不同的异常，可以尝试罗列多个 catch 语句，用于捕获不同类型的异常。

try/catch 案例

```
#include <iostream>
using namespace std;
double div(double x, double y) {
    return (x / y);
}
double add(double x, double y) {
    return x + y;
}
int main() {
    double v1 = 50, v2 = 0;
    double result1 = div(v1, v2);
    cout << result1 << endl;

    double result2 = add(v1, v2);
    cout << result2 << endl;

    return 0;
}
```

inf : infinity (linux) 等同于 #INF:infinity (windows)
nan : not a number 等同于 #IND:indeterminate (windows)

inf一般是因为得到的数值，超出浮点数的表示范围（溢出，即阶码部分超过其能表示的最大值）；而nan一般是因为对浮点数进行了未定义的操作，如对-1开方。

要想代码正常运行完，如何修改代码？

try/catch 案例

```
#include <iostream>
using namespace std;
double div(double x, double y) {
    return (x / y);
}
double add(double x, double y) {
    return x + y;
}
int main() {
    double v1 = 50, v2 = 0;
    double result1 = div(v1, v2);
    cout << result1 << endl;

    double result2 = add(v1, v2);
    cout << result2 << endl;

    return 0;
}
```

需要div函数里面判断除数是否为0，如果为0，
需要抛出异常。
如何抛出异常？

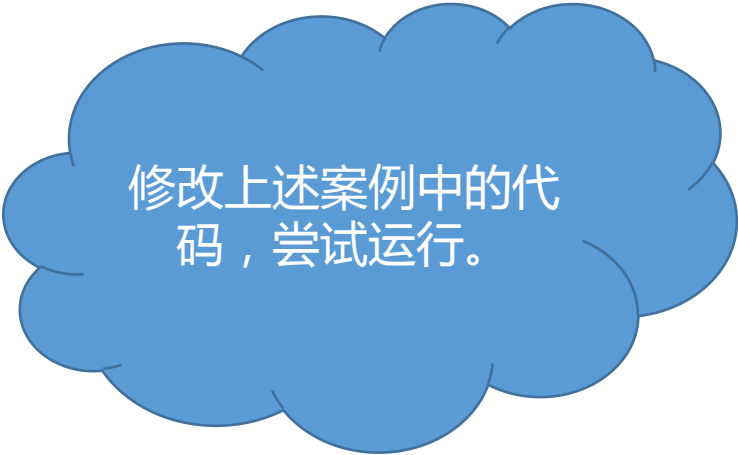
抛出异常

使用 throw 语句在代码块中的任何地方抛出异常。

throw 语句的操作数可以是任意的表达式，表达式的结果的类型决定了抛出的异常的类型。

以下是尝试除以零时抛出异常的实例：

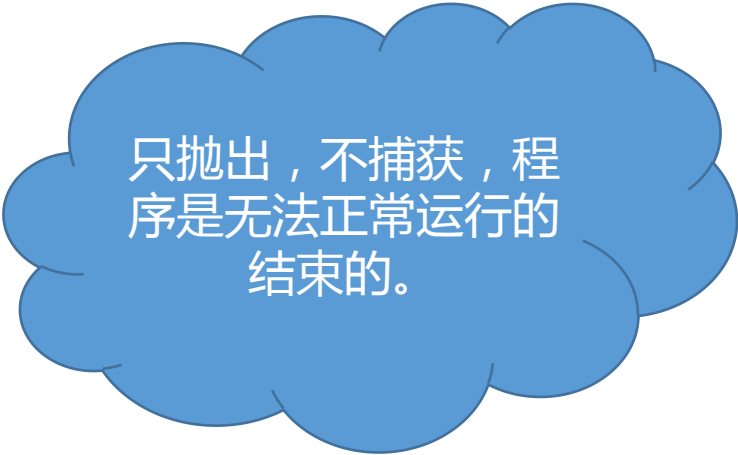
```
double div(double a, double b)
{
    if( b == 0 )
    {
        throw "Division by zero condition!";
    }
    return (a/b);
}
```



修改上述案例中的代码，尝试运行。

抛出异常

当在div函数里面抛出异常后，程序还是无法正常运行结束。



只抛出，不捕获，程序是无法正常运行的。
结束的。

捕获异常

catch 块跟在 try 块后面，用于捕获异常。可以指定想要捕捉的异常类型，这是由 catch 关键字后的括号内的异常声明决定的。

```
try
{
    // 保护代码
} catch( ExceptionName e )
{
    // 处理 ExceptionName 异常的代码
}
```

捕获一个类型为 ExceptionName 的异常

```
try
{
    // 保护代码
} catch(...)
{
    // 能处理任何异常的代码
}
```

让 catch 块能够处理 try 块抛出的任何类型的异常，则必须在异常声明的括号内使用省略号 ...

捕获异常

当给div函数加上try catch之后，程序可以正常运行结束了。

```
try {  
    double result1 = div(v1, v2);  
    cout << result1 << endl;  
}  
  
catch (...) {  
    cout << "出错了！" << endl;  
}
```

```
int main() {  
    double v1 = 50, v2 = 0;  
    try {  
        double result1 = div(v1, v2);  
        cout << result1 << endl;  
    }  
    catch (...) {  
        cout << "出错了！" << endl;  
    }  
}
```



Microsoft Visual Studio 调试控制台

出错了!
50

E:\UStudioCode\Test001\x64\De
按任意键关闭此窗口. . .

捕获异常

那如何打印刚才throw关键字抛出的异常信息呢？

```
try {  
    double result1 = div(v1, v2);  
    cout << result1 << endl;  
}  
catch (const char * msg) {  
    cerr << msg << endl;  
}
```

```
int main() {  
    double v1 = 50, v2 = 0;  
    try {  
        double result1 = div(v1, v2);  
        cout << result1 << endl;  
    }  
    catch (const char * msg) {  
        cerr << msg << endl;  
    }  
}
```



自定义异常

通过继承和重载 exception 类来定义新的异常。

```
#include <iostream>
```

```
#include <exception>
```

```
using namespace std;
```

```
class MyException : public exception {  
public:
```

```
    const char* what() const throw()
```

```
{
```

```
    return "my test exception";
```

```
}
```

```
};
```

what()是一个由 exception 类提供的公共方法。它用于返回异常的原因。

自定义异常

```
int add(int v1, int v2) {  
    if (v1 > 0) { //如果v1 > 0 ,那么抛出自定义异常myex  
        MyException myex;  
        throw myex;  
    }  
    return v1 + v2;  
}  
int main() {  
    int num1 = 100, num2 = 200;  
    try {  
        cout << add(num1, num2) << endl;  
    }  
    catch (exception& e) {  
        cout << e.what() << endl;  
    }  
    return 0;  
}
```

异常处理练习题

创建一个数组，创建一个函数取数组元素，当数组下标小于0,或者数组下标大于最大下标时抛出异常，其他正常取值。

下标小于0 **underflow**

下标大于n-1 **overflow**

异常处理 练习题代码

```
#include<iostream>
```

```
using namespace std;
```

```
enum index{underflow, overflow};//under下标小于0 , over下标超过最大下标
```

```
int array_index(int *arr, int n, int index);
```

异常处理 练习题代码

```
int main()
{
    int *arr = new int[10];
    for(int i=0; i<10; i++)
        A[i] = i;
    try
    {
        cout<<array_index(arr,10,5)<<endl;
        cout<<array_index(arr,10,-1)<<endl;
        cout<<array_index(arr,10,15)<<endl;
    }
    catch(index e)
    {
        if(e == underflow)
        {
            cout<<"index underflow!"<<endl;
            exit(-1);
        }
        if(e == overflow)
        {
            cout<<"index overflow!"<<endl;
            exit(-1);
        }
    }
    return 0;
}
```

异常处理 练习题代码

```
int array_index(int *arr, int n, int index)
{
    if(index < 0) throw underflow;
    if(index > n-1) throw overflow;
    return arr[index];
}
```

嵌套类和局部类

嵌套类

外围类需要使用嵌套类对象作为底层实现，并且该嵌套类只用于外围类的实现，且同时可以对用户隐藏该底层实现。

从作用域的角度看，嵌套类被隐藏在外围类之中，该类名只能在外围类中使用。如果在外围类之外的作用域使用该类名时，需要加名字限定。

嵌套类中的成员函数可以在它的类体外定义。

嵌套类的成员函数对外围类的私有成员没有访问权，反之亦然。

嵌套类仅仅只是语法上的嵌入。

注意：老版本的C++和部分C++编译器是不能使用嵌套类这一特性的。

嵌套类和局部类

嵌套类基本语法结构：

```
class 外围类名 { //类体
```

```
    class 嵌套类名 { //类体
```

```
    };
```

```
};
```

嵌套类和局部类

嵌套类的特点

1. 嵌套类不可以访问外围类的任何成员
2. 外围类可以通过对象访问嵌套类的公有成员，但不能访问保护和私有成员。
3. 嵌套类只能由外围类使用

嵌套类和局部类

局部类

类也可以定义在函数体内，这样的类被称为局部类。局部类只在定义它的局部域内可见。

局部类的成员函数必须被定义在类体中。

局部类中不能有静态成员函数。

在实践中，局部类很少使用。

嵌套类和局部类

局部类基本语法结构：

```
type funcName(parameter list) { // 函数  
    class 局部类名 { // 类体  
  
    };  
};
```

嵌套类和局部类 案例

```
#include <iostream>
using namespace std;
class OuterClass { //外部类
public:
    class InnerClass { //嵌套类
    public:
        void func(); //嵌套类的func方法
    };
public:
    InnerClass obj;
    void func() { //外部类的func方法
        cout << "OuterClass func()" << endl;
        obj.func();
    }
};
```

嵌套类和局部类 案例

```
void OuterClass::InnerClass::func() { //嵌套类func方法具体实现
    cout << "InnerClass func()" << endl;
}
void func() {
    class LocalClass { //局部类
    public:
        int num;
        void setNum(int n) {
            num = n;
        }
        void showNum() {
            cout << "num=" << num << endl;
        }
    };
    LocalClass lc;
    lc.setNum(111);
    lc.showNum();
}
```

嵌套类和局部类 案例

```
int main() {  
  
    OuterClass oc1;  
    oc1.func();  
    cout << "-----" << endl;  
  
    OuterClass::InnerClass ic1;  
    ic1.func();  
    cout << "-----" << endl;  
  
    func();  
  
    return 0;  
}
```

```
OuterClass func()  
InnerClass func()  
-----  
InnerClass func()  
-----  
num=111
```

知识点总结

- | | |
|----------------|----|
| 1. 异常处理概述 | 了解 |
| 2. 标准异常 | 熟悉 |
| 3. try/catch语句 | 重点 |
| 4. 抛出异常 | 重点 |
| 5. 捕获异常 | 重点 |
| 6. 自定义异常 | 重点 |
| 7. 嵌套类和局部类 | 熟悉 |

作业

(1) 什么叫异常处理?

(2) C++的异常处理机制有何优点?

作业

(1) 什么叫异常处理?

答：异常处理是一种程序定义的错误，它对程序的逻辑错误进行设防，并对运行异常加以控制。在C++中，异常处理是对所能预料的运行错误进行处理的一套实现机制。

(2) C++的异常处理机制有何优点?

答：C++的异常处理机制使得异常的引发和处理不必在同一函数中。C++异常处理的目的是在异常发生时，尽可能地减小破坏，周密地善后，而不影响其他部分程序的运行。这样底层的函数可以着重解决具体问题，而不必过多地考虑对异常的处理。上层调用者可以在适当的位置设计对不同类型异常的处理，这在大型程序中是非常必要的

下节课预告

模板